JOBSHEET V

Mata Kuliah: Praktikum Algoritma Dan Struktur Data

BRUTE FORCE DAN DIVIDE CONQUER

Dosen Pengampu:

Mungki Astiningrum, S.T., M.Kom.

Disusun oleh:

Ahmad Rafid Riqkullah

254107020078

T1 – 1G

D-IV Teknik Informatika

POLITEKNIK NEGERI MALANG

Jl. Soekarno Hatta No.9, Jatimulyo, Kec. Lowokwaru, Kota Malang, Jawa Timur

65141

TAHUN 2025-2026

5. Praktikum

5.2 Menghitung Nilai Faktorial dengan Algoritma Brute Force dan Divide and Conquer

5.2.1. Langkah-langkah Percobaan

- **Class Faktorial**

```
package Jobsheet5.BruteForceDivideConquer;


public class Faktorial {
    int faktorialBF(int n) {
        int faktorial = 1;
        for (int i = 1; i <= n; i++) {
            faktorial = faktorial * i;
        }
        return faktorial;
    }


    int faktorialDC(int n) {
        if (n == 1) {
            return 1;
        } else {
            int fakto = n * faktorialDC(n - 1);
            return fakto;
        }
    }
}
```

- **Class MainFaktorial**

```
package Jobsheet5.BruteForceDivideConquer;


import java.util.Scanner;
```

```
public class MainFaktorial {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        System.out.print("Masukkan nilai: ");
        int nilai = input.nextInt();

        Faktorial fk = new Faktorial();

        System.out.println("Nilai faktorial " + nilai + " menggunakan BF: " + fk.faktorialBF(nilai));
        System.out.println("Nilai faktorial " + nilai + " menggunakan DC: " + fk.faktorialDC(nilai));

        input.close();
    }
}
```

5.2.2. Verifikasi Hasil Percobaan

```
PS C:\Users\IdeaPad\OneDrive\Dokumen\SEMESTER 2\Algoritma &  Struktur Data\Praktik Algo&StrukturData>  & 'C:\Program
wCodeDetailsInExceptionMessages' '-cp' 'C:\Users\IdeaPad\AppData\Roaming\Code\User\workspaceStorage\125289802138902
k Algo&StrukturData_7951cb7c\bin' 'Jobsheet5.BruteForceDivideConquer.MainFaktorial'
Masukkan nilai: 5
Nilai faktorial 5 menggunakan BF: 120
Nilai faktorial 5 menggunakan DC: 120
PS C:\Users\IdeaPad\OneDrive\Dokumen\SEMESTER 2\Algoritma &  Struktur Data\Praktik Algo&StrukturData>
```

5.2.3. Pertanyaan

1. Pada base line Algoritma Divide Conquer untuk melakukan pencarian nilai faktorial, jelaskan perbedaan bagian kode pada penggunaan if dan else!
   Jawab:
   - **Bagian if (n == 1):** Ini adalah kondisi berhenti (*base case*). Artinya, ketika nilai n sudah mencapai angka 1, proses pemanggilan fungsi akan dihentikan dan program langsung mengembalikan nilai 1.
   - **Bagian else:** Ini adalah bagian di mana fungsi memanggil dirinya sendiri (*rekursif*). Di sini, program akan terus memecah perkalian dengan nilai yang semakin mengecil (n - 1) sampai akhirnya menyentuh kondisi if di atas.

2. Apakah memungkinkan perulangan pada method faktorialBF() diubah selain menggunakan for? Buktikan!
   Jawab:
   Sangat memungkinkan. Kita bisa mengubah perulangan for menjadi perulangan while atau do-while. Logika perhitungannya tetap sama, hanya cara penulisan perulangannya yang berbeda.

```
int faktorialBF(int n) {
        int faktorial = 1;
        int i = 1;

        while (i <= n) {
            faktorial = faktorial * i;
            i++;
        }
        return faktorial;
    }
```

Buktinya, Pada fungsi faktorialBF ubah for menjadi menggunakan while, dan hasilnya akan tetap sama.

```
Masukkan nilai: 5
Nilai faktorial 5 menggunakan BF: 120
Nilai faktorial 5 menggunakan DC: 120
PS C:\Users\IdeaPad\OneDrive\Dokumen\SEMESTER 2\Algoritma &  Struktur Data\Praktik Algo&StrukturData>
```

3. Jelaskan perbedaan antara fakto *= i; dan int fakto = n * faktorialDC(n-1);!
   Jawab:
   - **fakto *= i** (atau faktorial = faktorial * i): Ini adalah perhitungan menggunakan perulangan (iteratif). Nilai sebelumnya akan terus dikalikan dengan variabel i yang nilainya terus bertambah satu per satu sampai perulangan selesai.
   - **int fakto = n * faktorialDC(n-1);**: Ini adalah perhitungan dengan cara memanggil fungsinya sendiri (rekursif). Di sini, program mengalikan angka saat ini (n) dengan hasil pemanggilan fungsi faktorial dari angka sebelumnya (n-1).

4. Buat Kesimpulan tentang perbedaan cara kerja method faktorialBF() dan faktorialDC()!
   Jawab:
   - **faktorialBF() (Brute Force):** Menggunakan cara iteratif (perulangan biasa). Program menghitung perkalian secara berurutan dan maju, mulai dari angka 1 terus naik sampai mencapai angka n.
   - **faktorialDC() (Divide and Conquer):** Menggunakan cara rekursif (memanggil fungsinya sendiri). Program memecah masalah dari angka yang paling besar (n) menghitung mundur sampai angka 1, kemudian menggabungkan hasil perkaliannya menjadi satu nilai akhir.

5.3 Menghitung Hasil Pangkat dengan Algoritma Brute Force dan Divide and Conquer

5.3.1. Langkah-langkah Percobaan

- **Class Pangkat**

```
package Jobsheet5.BruteForceDivideConquer;

public class pangkat {
    public int nilai, pangkat;

    public pangkat(int n, int p) {
        nilai = n;
        pangkat = p;
    }

    public int pangkatBF(int a, int n) {
        int hasil = 1;
        for (int i = 0; i < n; i++) {
            hasil = hasil * a;
        }
        return hasil;
    }

    public int pangkatDC(int a, int n) {
        if (n == 1) {
            return a;
        } else {
            if (n % 2 == 1) { // Jika pangkat ganjil
                return (pangkatDC(a, n / 2) * pangkatDC(a, n / 2) * a);
            } else { // Jika pangkat genap
                return (pangkatDC(a, n / 2) * pangkatDC(a, n / 2));
            }
        }
    }
}
```

- **Class MainPangkat**

```
package Jobsheet5.BruteForceDivideConquer;


import java.util.Scanner;


public class MainPangkat {

    public static void main(String[] args) {

        Scanner input = new Scanner(System.in);
```

```
        System.out.print("Masukkan jumlah elemen: ");
        int elemen = input.nextInt();

        pangkat[] png = new pangkat[elemen];
        for (int i = 0; i < elemen; i++) {
            System.out.print("Masukkan nilai basis elemen ke-" + (i + 1) + ": ");
            int basis = input.nextInt();
            System.out.print("Masukkan nilai pangkat elemen ke-" + (i + 1) + ": ");
            int pangkat = input.nextInt();

            png[i] = new pangkat(basis, pangkat);
        }

System.out.println("==========================================");
        System.out.println("HASIL PANGKAT BRUTEFORCE:");
        for (pangkat p : png) {
            System.out.println(p.nilai + "^" + p.pangkat + ": " + p.pangkatBF(p.nilai,
p.pangkat));
        }

System.out.println("==========================================");
        System.out.println("HASIL PANGKAT DIVIDE AND CONQUER:");
        for (pangkat p : png) {
            System.out.println(p.nilai + "^" + p.pangkat + ": " + p.pangkatDC(p.nilai,
p.pangkat));
        }

        input.close();
    }
}
```

5.3.2. Verifikasi Hasil Percobaan

```
PS C:\Users\IdeaPad\OneDrive\Dokumen\SEMESTER 2\Algoritma &  Struktur Data\Praktik Algo&StrukturData>  & 'C:\Pr
a.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\IdeaPad\AppData\Roaming\Code\User\workspaceSto
7124bb\redhat.java\jdt_ws\Praktik Algo&StrukturData_7951cb7c\bin' 'Jobsheet5.BruteForceDivideConquer.MainPangka
Masukkan jumlah elemen: 3
Masukkan nilai basis elemen ke-1: 2
Masukkan nilai pangkat elemen ke-1: 3
Masukkan nilai basis elemen ke-2: 4
Masukkan nilai pangkat elemen ke-2: 5
Masukkan nilai basis elemen ke-3: 6
Masukkan nilai pangkat elemen ke-3: 7
========================================
HASIL PANGKAT BRUTEFORCE:
2^3: 8
4^5: 1024
6^7: 279936
========================================
HASIL PANGKAT DIVIDE AND CONQUER:
2^3: 8
4^5: 1024
6^7: 279936
PS C:\Users\IdeaPad\OneDrive\Dokumen\SEMESTER 2\Algoritma &  Struktur Data\Praktik Algo&StrukturData>
```

5.3.3. Pertanyaan

1. Jelaskan mengenai perbedaan 2 method yang dibuat yaitu pangkatBF() dan pangkatDC()!
   Jawab:
   - **pangkatBF():** Menggunakan pendekatan **iteratif** (perulangan biasa). Method ini mengalikan angka (basis) secara berurutan satu per satu sebanyak nilai pangkatnya menggunakan perulangan for.
   - **pangkatDC():** Menggunakan pendekatan **rekursif** (memanggil fungsinya sendiri). Method ini membagi masalah menjadi lebih kecil dengan membagi nilai pangkat menjadi dua (n/2) secara terus-menerus, kemudian hasil potongannya dikalikan kembali untuk mendapatkan hasil akhir.

2. Apakah tahap combine sudah termasuk dalam kode tersebut?Tunjukkan!
   Jawab:
   Ya, tahap combine atau tahap penggabungan sudah termasuk di dalam kode. Tahap ini terjadi ketika program menggabungkan hasil dari pemecahan masalah rekursif. Hal ini ditunjukkan pada baris kode di dalam method pangkatDC():
   - Untuk pangkat ganjil: return (pangkatDC(a, n / 2) * pangkatDC(a, n / 2) * a);
   - Untuk pangkat genap: return (pangkatDC(a, n / 2) * pangkatDC(a, n / 2));

   Tanda bintang (*) yang berfungsi mengalikan hasil pemanggilan fungsi, itulah yang merupakan proses combine.

3. Pada method pangkatBF()terdapat parameter untuk melewatkan nilai yang akan dipangkatkan dan pangkat berapa, padahal di sisi lain di class Pangkat telah ada atribut nilai dan pangkat, apakah menurut Anda method tersebut tetap relevan untuk memiliki parameter? Apakah bisa jika method tersebut dibuat dengan tanpa parameter? Jika bisa, seperti apa method pangkatBF() yang tanpa parameter?
   Jawab:
   Sebenarnya method tersebut relevan relevan saja untuk menggunakan parameter. Tetapi menurut saya tidak memakai parameter pun sudah cukup, karena mau ada parameter ataupun tidak program masi tetap bisa menghitung pada fungsi tsb.
   Sangat bisa jika method tersebut dibuat tanpa parameter.

```java
public int pangkatBF() {
        int hasil = 1;
        for (int i = 0; i < pangkat; i++) {
            hasil = hasil * nilai;
        }
        return hasil;
    }
```

Hilangkan parameter di class pangkat di fungsi pangkatBF, lalu di main jangan lupa di hilangkan juga parameternya.

```java
System.out.println("=====================================");
        System.out.println("HASIL PANGKAT BRUTEFORCE:");
        for (pangkat p : png) {
            System.out.println(p.nilai + "^" + p.pangkat + ": " + p.pangkatBF());
        }
```

   Hasilnya akan tetap sama

```
ptionMessages' '-cp' 'C:\Users\IdeaPad\AppData\Roamin
Algo&StrukturData_7951cb7c\bin' 'Jobsheet5.BruteForce
Masukkan jumlah elemen: 3
Masukkan nilai basis elemen ke-1: 2
Masukkan nilai pangkat elemen ke-1: 3
Masukkan nilai basis elemen ke-2: 4
Masukkan nilai pangkat elemen ke-2: 5
Masukkan nilai basis elemen ke-3: 6
Masukkan nilai pangkat elemen ke-3: 7
=====================================
HASIL PANGKAT BRUTEFORCE:
2^3: 8
4^5: 1024
6^7: 279936
=====================================
HASIL PANGKAT DIVIDE AND CONQUER:
2^3: 8
4^5: 1024
6^7: 279936
```

4. Tarik tentang cara kerja method pangkatBF() dan pangkatDC()!
   Jawab:
   - **Cara kerja pangkatBF():** Bekerja secara lurus dan langsung dengan cara mengalikan angka basis berulang kali sesuai dengan jumlah

pangkatnya. Langkah perhitungannya akan sama banyaknya dengan nilai pangkat itu sendiri.

- **Cara kerja pangkatDC():** Bekerja dengan membagi nilai pangkat menjadi dua bagian secara berulang (Divide) hingga mencapai nilai terkecil, lalu hasil perhitungannya dikalikan kembali menjadi satu (Conquer & Combine). Pendekatan ini membuat proses perhitungan menjadi lebih efisien.

5.4 Menghitung Sum Array dengan Algoritma Brute Force dan Divide and Conquer

5.4.1. Langkah-langkah Percobaan

- **Class Sum**

```
package Jobsheet5.BruteForceDivideConquer;

public class sum {
    public double keuntungan[];

    public sum(int el) {
        keuntungan = new double[el];
    }

    public double totalBF() {
        double total = 0;
        for (int i = 0; i < keuntungan.length; i++) {
            total = total + keuntungan[i];
        }
        return total;
    }

    public double totalDC(double arr[], int l, int r) {
        if (l == r) {
            return arr[l];
        }

        int mid = (l + r) / 2;
        double lsum = totalDC(arr, l, mid);
        double rsum = totalDC(arr, mid + 1, r);
        return lsum + rsum;
    }
}
```

- **Class MainSum**

```
package Jobsheet5.BruteForceDivideConquer;
```

```
import java.util.Scanner;

public class mainSum {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);

        System.out.print("Masukkan jumlah elemen (bulan): ");
        int elemen = input.nextInt();

        sum sm = new sum(elemen);
        for (int i = 0; i < elemen; i++) {
            System.out.print("Masukkan keuntungan bulan ke-" + (i + 1) + ": ");
            sm.keuntungan[i] = input.nextDouble();
        }

System.out.println("==========================================================");
        System.out.println("Total keuntungan menggunakan Brute Force        : " + sm.totalBF());
        System.out.println("Total keuntungan menggunakan Divide and Conquer: " + sm.totalDC(sm.keuntungan, 0, elemen - 1));

System.out.println("==========================================================");

        input.close();
    }
}
```

5.4.2. Verifikasi Hasil Percobaan

```
PS C:\Users\IdeaPad\OneDrive\Dokumen\SEMESTER 2\Algoritma &  Struktur Data\Praktik Algo&StrukturData>  & 'C:\Program F
a.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\IdeaPad\AppData\Roaming\Code\User\workspaceStorage\12
7124bb\redhat.java\jdt_ws\Praktik Algo&StrukturData_7951cb7c\bin' 'Jobsheet5.BruteForceDivideConquer.mainSum'
Masukkan jumlah elemen (bulan): 5
Masukkan keuntungan bulan ke-1: 10
Masukkan keuntungan bulan ke-2: 20
Masukkan keuntungan bulan ke-3: 30
Masukkan keuntungan bulan ke-4: 40
Masukkan keuntungan bulan ke-5: 50
============================================================
Total keuntungan menggunakan Brute Force       : 150.0
Total keuntungan menggunakan Divide and Conquer: 150.0
============================================================
PS C:\Users\IdeaPad\OneDrive\Dokumen\SEMESTER 2\Algoritma &  Struktur Data\Praktik Algo&StrukturData>
```

5.4.3. Pertanyaan

1. Kenapa dibutuhkan variable mid pada method TotalDC()?
   Jawab:
   Variabel mid fungsinya sebagai titik potong untuk membelah array jadi dua bagian (kiri dan kanan). Ini adalah tahap divide (membagi), tujuannya supaya masalah yang besar bisa dipecah jadi potongan-potongan kecil yang sama rata.

2. Untuk apakah statement di bawah ini dilakukan dalam TotalDC()?

   ```
   double lsum = totalDC(arr, l, mid);
   double rsum = totalDC(arr, mid+1, r);
   ```

   Jawab:

   Statement ini digunakan untuk menyelesaikan masalah yang sudah dipecah tadi secara rekursif conquer:

   - **lsum (left sum):** Memanggil fungsinya sendiri untuk mencari total nilai array di belahan kiri (dari indeks l awal sampai ke tengah mid).
   - **rsum (right sum):** Memanggil fungsinya sendiri untuk mencari total nilai array di belahan kanan (dari indeks setelah tengah mid + 1 sampai ke akhir r).

3. Kenapa diperlukan penjumlahan hasil lsum dan rsum seperti di bawah ini?

   ```
   return lsum+rsum;
   ```

   Jawab:
   Penjumlahan ini adalah tahap Combine atau menggabungkan. Karena array awalnya dipecah dan dihitung secara terpisah menjadi bagian kiri (lsum) dan kanan (rsum) dan untuk mendapatkan nilai total keseluruhan yang sebenarnya, kedua hasil tersebut harus dijumlahkan kembali.

4. Apakah base case dari totalDC() ?
   Jawab:
   Base case-nya adalah kondisi if (l == r). Artinya, proses membagi array akan berhenti kalau batas kiri (l) dan kanan (r) sudah sama, yang berarti elemennya cuma sisa satu. Jika sudah begini, program langsung mengambil nilai elemen tunggal tersebut (return arr[l];).

5. Tarik Kesimpulan tentang cara kerja totalDC()
   Jawab:
   Kesimpulannya, method totalDC() bekerja dengan menerapkan tiga langkah utama *Divide and Conquer*:
   1. Divide (Membagi): Membelah array berulang kali menjadi dua bagian menggunakan nilai mid sampai hanya tersisa satu elemen tunggal (*base case*).
   2. Conquer (Menyelesaikan): Menghitung masing-masing sub-masalah secara terpisah dengan cara memanggil fungsinya sendiri (rekursif) untuk bagian kiri (lsum) dan kanan (rsum).
   3. Combine (Menggabungkan): Menjumlahkan kembali hasil potongan-potongan tersebut (lsum + rsum) dari bawah ke atas hingga mendapatkan nilai total akhir dari seluruh elemen array.

4.5 Latihan Praktikum

1. Sebuah kampus memiliki daftar nilai mahasiswa dengan data sesuai tabel di bawah ini

Nama	NIM	Tahun Masuk	Nilai UTS	Nilai UAS
Ahmad	220101001	2022	78	82
Budi	220101002	2022	85	88
Cindy	220101003	2021	90	87
Dian	220101004	2021	76	79
Eko	220101005	2023	92	95
Fajar	220101006	2020	88	85
Gina	220101007	2023	80	83
Hadi	220101008	2020	82	84

Tentukan:

a) Nilau UTS tertinggi tertinggi menggunakan Divide and Conquer!

b) Nilai UTS terendah menggunakan Divide and Conquer!

c) Rata-rata nilai UAS dari semua mahasiswa menggunakan Brute Force!

Code mahasiswa:

```
package Jobsheet5.BruteForceDivideConquer;


public class Mahasiswa {
    public String nama;
    public int nim;
    public int tahunMasuk;
    public int nilaiUTS;
    public int nilaiUAS;


    public Mahasiswa(String nama, int nim, int tahunMasuk, int nilaiUTS, int nilaiUAS) {
        this.nama = nama;
        this.nim = nim;
        this.tahunMasuk = tahunMasuk;
        this.nilaiUTS = nilaiUTS;
        this.nilaiUAS = nilaiUAS;
    }
}
```

Code mainMahasiswa:

```
package Jobsheet5.BruteForceDivideConquer;


public class mainMahasiswa {
    public static void main(String[] args) {
        Mahasiswa[] mhs = new Mahasiswa[8];


        mhs[0] = new Mahasiswa("Ahmad", 220101001, 2022, 78, 82);
        mhs[1] = new Mahasiswa("Budi",  220101002, 2022, 85, 88);
        mhs[2] = new Mahasiswa("Cindy", 220101003, 2021, 90, 87);
        mhs[3] = new Mahasiswa("Dian",  220101004, 2021, 76, 79);
```

```
        mhs[4] = new Mahasiswa("Eko",   220101005, 2023, 92, 95);
        mhs[5] = new Mahasiswa("Fajar", 220101006, 2020, 88, 85);
        mhs[6] = new Mahasiswa("Gina",  220101007, 2023, 80, 83);
        mhs[7] = new Mahasiswa("Hadi",  220101008, 2020, 82, 84);
        System.out.println("==============================================");
        System.out.println("a) Nilai UTS Tertinggi (DC) : " + cariMaxUTS(mhs, 0, mhs.length - 1));
        System.out.println("b) Nilai UTS Terendah (DC) : " + cariMinUTS(mhs, 0, mhs.length - 1));
        System.out.println("c) Rata-rata UAS (BF)       : " + rataRataUAS(mhs));
        System.out.println("==============================================");
    }
    // nilai max pakai divide and conquer
    public static int cariMaxUTS(Mahasiswa[] arr, int l, int r) {
        if (l == r) {
            return arr[l].nilaiUTS;
        } else {
            int mid = (l + r) / 2;
            int lMax = cariMaxUTS(arr, l, mid);
            int rMax = cariMaxUTS(arr, mid + 1, r);
            return (lMax > rMax) ? lMax : rMax; // Cara singkat if-else
        }
    }
    // niali min pakai divide and conquer
    public static int cariMinUTS(Mahasiswa[] arr, int l, int r) {
        if (l == r) {
            return arr[l].nilaiUTS;
        } else {
            int mid = (l + r) / 2;
```

```
            int lMin = cariMinUTS(arr, l, mid);
            int rMin = cariMinUTS(arr, mid + 1, r);
            return (lMin < rMin) ? lMin : rMin;
        }
    }
    // hitung Rata-rata UAS ini brute force
    public static double rataRataUAS(Mahasiswa[] arr) {
        double total = 0;
        for (int i = 0; i < arr.length; i++) {
            total += arr[i].nilaiUAS;
        }
        return total / arr.length;
    }
}
```

Hasil:

```
PS C:\Users\IdeaPad\OneDrive\Dokumen\SEMESTER 2\Algoritma &  Struktur Data\Praktik Algo&StrukturData>  & '
a.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\IdeaPad\AppData\Roaming\Code\User\workspa
7124bb\redhat.java\jdt_ws\Praktik Algo&StrukturData_7951cb7c\bin' 'Jobsheet5.BruteForceDivideConquer.mainM
================================================
a) Nilai UTS Tertinggi (DC) : 92
b) Nilai UTS Terendah (DC)  : 76
c) Rata-rata UAS (BF)       : 85.375
================================================
PS C:\Users\IdeaPad\OneDrive\Dokumen\SEMESTER 2\Algoritma &  Struktur Data\Praktik Algo&StrukturData>
```